Project Development Prompts

Prompt: I want to build a project called Maritime Route Risk Monitor. The idea is to monitor vessel activity in the North Sea using live AIS data, combine vessel positions with weather conditions, identify vessels that may be exposed to challenging weather, and show the information in a dashboard. I am new to some of the technologies that may be needed, so please first help me understand the problem and propose a suitable end-to-end architecture. I want the system to have separate parts for collecting data, moving data between services, processing it, storing it, analysing risk, analysing port activity, and displaying the results. Explain the purpose of each part in simple terms before recommending tools. Please do not start building the entire application at once. Break the work into stages and explain what should be completed and tested at each stage.

Prompt: Based on the project idea, help me choose a practical technology stack that I can run and demonstrate as a student project. I want Kafka to be part of the data pipeline between the data-producing and data-consuming components. I also need a way to process streaming data, store the results, retrieve live vessel information, obtain weather information, and build an interactive dashboard. Explain why each suggested technology is needed, how the components communicate, and what I need to install. I am learning these tools, so avoid assuming that I already understand their concepts. Please identify any prerequisites and potential compatibility issues before we begin implementation.

Prompt: Help me prepare my computer for this project. Guide me through installing and checking the required programming language, development tools, Java if it is needed for the streaming processor, Docker, and Git. Help me create a project folder and a Python virtual environment, install the initial dependencies, and organize the application into separate modules. Give me one task at a time, explain the commands before asking me to run them, and wait for me to confirm the result before moving to the next task. Do not assume a command succeeded just because I ran it.

Prompt: I want the application to receive live vessel-position information from AISStream. Please explain how the AIS data source works and what information is normally available in its messages. Help me obtain and configure the API key securely, then build a small first version that connects to the AISStream WebSocket and prints a few received messages so I can understand their structure. Once the connection works, help me extract the vessel identifier, position, timestamp, and other useful movement information when those fields are available. The connection should continue running and should attempt to reconnect if the connection is interrupted. Keep the API key out of the source code and explain how to store it safely.

Prompt: Now that I can receive AIS messages, help me turn them into a consistent format that the rest of the application can use. I want each event to contain a unique identifier, the vessel identifier, a normalized timestamp, valid position and movement information, and the original message where it is useful for troubleshooting. Please explain what should happen when a message is missing a field or contains an invalid coordinate or value. I want to avoid sending malformed records into the rest of the pipeline. Recommend a simple event structure, explain the decisions, and show me how to test it with actual messages received from the AIS source.

Prompt: I want to enrich the vessel observations with weather conditions that may affect maritime operations. Please investigate how I can use the Open-Meteo Marine API and forecast API for this purpose. Help me retrieve wave height, wind speed, wind gusts, and ocean-current velocity where the APIs provide them. Explain how to match weather information to the time of an AIS observation so that the application does not attach an unrelated or outdated forecast to a vessel eventHelp me assign each vessel

observation to the nearest reference location and explain the limitations of this approximation. Weather information should refresh periodically, around every 15 minutes, and old cached data should not be used beyond 30 minutes. Please make missing weather information visible rather than treating it as safe conditions.

Prompt: I want Kafka to be the messaging layer between the part of the application that receives vessel data and the part that processes it. I am new to Kafka, so explain producers, topics, partitions, consumers, and brokers using this project as the example. Help me run Kafka locally with Docker Compose, create a topic for vessel events, and verify that I can send and receive a test message. After the basic test works, help me connect the AIS event producer to Kafka. Use the vessel identifier as the message key where appropriate, and explain the producer settings that help protect messages and handle temporary broker problems. Keep the configuration in one place so it is easy to maintain.

Prompt: One of my most important requirements is that data should not disappear simply because Kafka or another part of the application goes down. If the AIS connection is still receiving messages but Kafka is temporarily unavailable, I want the application to preserve those received messages and send them after Kafka becomes available again. Please recommend a practical approach and explain its trade-offs before implementing it. I want a durable local queue or spool that saves each event before the application attempts to publish it. The system should track which events are still waiting and which have been successfully acknowledged by Kafka. Help me implement this carefully and test that pending events remain available after restarting the application.

Prompt: Please examine the failure cases in the data pipeline. For example, the publisher might send a message successfully to Kafka and then crash before updating the local queue. When it starts again, it may send the same message another time. I do not want this to create duplicate records in the final analytics database. Help me design the event identifiers and database-writing process so that repeated delivery of the same event can be recognized and handled safely. Explain the difference between preventing duplicates in the local queue, Kafka delivery, and the final database. I want reliable at-least-once delivery with duplicate-safe processing rather than an unsupported claim that every part of the system is exactly-once.

Prompt: Now that vessel events can be published to Kafka, help me build the consumer and processing component using Spark Structured Streaming. Explain how Spark reads messages from Kafka and how it processes them in small batches. Help me parse the event data, validate the fields needed for analysis, and write the processed observations to a separate analytics database. I want the processor to use a persistent checkpoint so it can remember its progress when it restarts. Please explain where the checkpoint should be stored and how it affects restarting the application. Keep the processor separate from the AIS ingestion and Kafka publisher code.

Prompt: I also want the processing component to recover when Kafka becomes unavailable or the streaming query fails. I do not want to depend on manually restarting the processor every time there is a temporary interruption. Please inspect the current Spark implementation and suggest a controlled retry or restart approach. It should retain the existing checkpoint and resume from the saved processing progress when the required services are available again. Add clear logs so I can see when the processor fails, waits, retries, and successfully starts again. Help me test this by temporarily stopping Kafka, observing the failure, starting Kafka again, and checking whether processing resumes. Do not delete or replace the checkpoint during this test.

Prompt: Help me create a separate database for the processed vessel observations that the dashboard and analytics modules will use. I want the database structure to support vessel positions, timestamps, weather information, and the fields required for risk and port analysis. The database should use the event identifier to recognize records that have already been stored. If Spark processes a message again after a restart, the database should not create another copy of the same event. Please explain the database design and implement the write operations in a way that is safe for repeated batches. Keep this analytics database separate from the local queue used to protect incoming events.

Prompt: One of the main business features is to give users an indication of the weather conditions that vessels are exposed to. I want the application to consider wave height, wind speed, wind gusts, and current velocity. I do not yet know the best way to turn these measurements into a score, so please explain a simple and transparent approach that is suitable for an academic prototype. I want the factors to contribute to an overall score and the result to be grouped into Low, Moderate, and High categories. Please recommend reasonable provisional thresholds, explain the assumptions and limitations, and make sure the system can report when there is not enough weather information to calculate a score. Do not present this as a certified safety assessment or an accident prediction model.

Prompt: I want to add a business feature that gives an overview of vessel activity around selected North Sea ports. Help me identify a practical way to use the AIS observations already stored in the analytics database to estimate when vessels enter or leave monitored port areas and to identify movements between ports. If official port boundary data is not available for this prototype, explain how approximate geofences could be used and what their limitations are. I want summaries such as observed arrivals, vessel counts, activity trends, frequently visited ports, and port-to-port transitions. Please account for gaps in AIS observations so that the system does not infer movements too confidently. Label the results as AIS-derived estimates, not official port-call statistics.

Prompt: I now want to present the processed information in an interactive dashboard using Streamlit. Help me design a clear layout for maritime monitoring. It should include key vessel metrics, a North Sea map showing vessel positions, a view of the weather conditions, the distribution of weather-exposure categories, and the port activity analysis. Users should be able to select vessels and filter by risk category. I want a vessel details view as well. Please build the dashboard using the modules and database already created rather than putting all the processing logic directly into the dashboard. Explain how each dashboard section obtains its data.

Prompt: The map should focus on the North Sea and make it easy to explore the monitored vessels. Help me set an appropriate map extent, centre, and zoom level. I want the filters to be useful without making the map jump or reset unexpectedly. Please make filter changes deliberate, so users can choose vessel and risk filters and then apply them. The map should not be constantly replaced while someone is hovering over a vessel, panning, or zooming. Keep the map interaction separate from the parts of the dashboard that need frequent updates.

Prompt: The dashboard needs current information, but refreshing the whole page every few seconds makes the map difficult to use and interrupts filter selection. Please improve the refresh design. The live vessel metrics, risk distribution, and weather measurements should update automatically without forcing the map to rerender. Port traffic analytics can refresh less frequently because it does not need the same update rate. Keep the map and its selected filters stable until the user chooses to apply filters or explicitly refreshes the map. Put the Refresh Map button below the map. Explain the changes and verify that each dashboard section refreshes independently.

Prompt: Help me test the system from the AIS source through to the dashboard. I want to verify that live AIS messages are received, saved to the local spool, published to Kafka, consumed by Spark, and written to the analytics database. Then help me test temporary failures by stopping Kafka while ingestion continues, checking that received events remain in the spool, restarting Kafka, and confirming that the publisher sends the pending events. Also test what happens if the Spark processor stops and starts again with its checkpoint. Check whether replayed events create duplicate analytics records. Please help me record the actual results, evidence, and limitations of each test. Do not claim that the system can recover AIS messages that were never delivered by the external provider.